

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	05/08/2026	DATE SUBMITTED:	05/08/2026
NAME:	EUNICE E. MORADILLO	INSTRUCTOR:	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

ILO A	
MAIN.CPP <pre>#include <iostream> #include <queue> void display(std::queue<char> copyQ); int main(){ //create an object std::queue<char> myQ; //use the enqueue operation myQ.push('E'); myQ.push('U'); myQ.push('N'); myQ.push('I'); std::cout<<"The current back is: "<<myQ.front()<<std::endl; std::cout<<"The current back is: "<<myQ.back()<<std::endl; display(myQ); //use the dequeue operation myQ.pop(); display(myQ); myQ.pop(); display(myQ); myQ.pop(); std::cout<<"After dequeue. the front is: "<<myQ.front()<<std::endl; //check if queue empty std::cout<<"is the queue empty? "<<myQ.empty()<<std::endl; return 0; } //note only use the member functions of the queue STL void display(std::queue<char> copyQ){ //create a copy of the queue std::queue<char> temp = copyQ; //loop until empty while (!temp.empty()){ //display the front std::cout<<" "<<temp.front()<<" "; //display the back temp.pop(); } //add a new line std::cout <<std::endl; }</pre>	CODE ANALYSIS: In this code, we first did to create the <code>std::queue<char></code> that uses <code>push()</code> to enqueue the letters E, U, N, I in that order. Since queue is a container adaptor built on deque by default, <code>push()</code> always inserts at the back while <code>front()/back()</code> give read-only access to the two ends without removing anything. The <code>display()</code> function takes the queue by value, so a copy (c) is made inside it. Looping <code>pop()</code> on the copy leaves myQ itself untouched. The <code>pop()</code> calls in <code>main()</code> remove elements strictly from the front, preserving FIFO order.
OUTPUT <pre>The current back is: E The current back is: I E U N I U N I N I After dequeue. the front is: I is the queue empty? 0</pre>	ANALYSIS: In the output, it showed that <code>front()</code> returns E because E was the first item enqueued, and <code>back()</code> returns I because it was the most recently pushed — this confirms the front/back pointers are working as expected for FIFO. The three <code>display()</code> calls show the queue removing one element at a time from the front,

proving that pop() always removes the oldest remaining item rather than the newest. After three pops only I is left, so front() correctly returns I. Finally, empty() returns 0 (false) because one element (I) is still in the queue.

OBSERVATION

From what I've observed is that the push() is always added to the back and pop() is always removed from the front, so the FIFO order was never broken. front() and back() only let us look at the two ends without removing anything, so the queue's size only changed when pop() was actually called. The display() function had to work on a copy of the queue instead of the original, since STL queues don't allow looping through their elements directly.

ILO B

MAIN.CPP

```
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout << "=====\\n";
    std::cout << "          QUEUE OPERATIONS          \\n";
    std::cout << "=====\\n";
    std::cout << "I Enqueuing letters: E U N I C E \\n";
    enqueue('E', &front, &back);
    enqueue('U', &front, &back);
    enqueue('N', &front, &back);
    enqueue('I', &front, &back);
    enqueue('C', &front, &back);
    enqueue('E', &front, &back);
    std::cout << "Current Queue:\\n";
    displayAll(&front, &back);
    std::cout << "  Front -> " << front->data << std::endl;
    std::cout << "  Back  -> " << back->data << std::endl;
    std::cout << "  Empty? " << (isEmpty(&front, &back) ? "Yes" : "No") << std::endl;

    std::cout << "=====\\n";
    std::cout << "          DEQUEUEING ELEMENTS          \\n";
    std::cout << "=====\\n";

    dequeue(&front, &back);
    displayAll(&front, &back);

    dequeue(&front, &back);
    displayAll(&front, &back);

    dequeue(&front, &back);
    displayAll(&front, &back);

    dequeue(&front, &back);
    displayAll(&front, &back);

    dequeue(&front, &back);
    displayAll(&front, &back);

    std::cout << "\\nIs Empty? " << (isEmpty(&front, &back) ? "Yes" : "No") << std::endl;

    // Test dequeue on empty queue
    dequeue(&front, &back);

    return 0;
}
```

CODE ANALYSIS:

In this code, we use the function main() that uses node pointers front and back (both initialized to nullptr) rather than a queue object, and passes them by address into free functions (enqueue, dequeue, displayAll, isEmpty) so those functions can modify the caller's pointers directly. The letters of "EUNICE" are enqueued one at a time, then the queue is dequeued repeatedly with a displayAll() call after each step to visualize the front shrinking.

QUEUELL.H

CODE ANALYSIS:

In the header file program, we used different functions to call it to the main.cpp. Enqueue() allocates a new node and links it after the current backPtr, updating backPtr to point to the new node, where in a special case handles the very first insertion where both frontPtr and backPtr are still nullptr. Dequeue() saves

```
#ifndef QUEUE_H
#define QUEUE_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};

// Create a new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>();
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// Enqueue
template <typename T>
void enqueue(T newData, qNode<T>* frontPtr, qNode<T>* backPtr) {
    qNode<T>* newNode = new_node(newData);

    if ((*frontPtr == nullptr) && (*backPtr == nullptr)) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

// Dequeue
template <typename T>
void dequeue(qNode<T>* frontPtr, qNode<T>* backPtr) {
    if ((*frontPtr == nullptr) && (*backPtr == nullptr)) {
        std::cout << "The queue is empty." << std::endl;
        return;
    }

    qNode<T>* deleteTemp = *frontPtr;

    // Only one node
    if ((*frontPtr)->next == nullptr) {
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteTemp;
        return;
    }

    *frontPtr = (*frontPtr)->next;
    delete deleteTemp;
}

// Check if queue is empty
template <typename T>
bool isEmpty(qNode<T>* frontPtr, qNode<T>* backPtr) {
    return ((*frontPtr == nullptr) && (*backPtr == nullptr));
}

// Display all elements
template <typename T>
void displayAll(qNode<T>* frontPtr, qNode<T>* backPtr) {
    if (isEmpty(frontPtr, backPtr)) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    qNode<T>* temp = *frontPtr;

    std::cout << "Queue Elements: ";

    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }

    std::cout << std::endl;
}

#endif
```

frontPtr in a temp pointer, advances frontPtr to the next node, and deletes the temp. isEmpty() simply checks whether frontPtr is nullptr.

OUTPUT

```
=====
                QUEUE OPERATIONS
=====
[ Enqueuing letters: E U N I C E ]
Current Queue:
Queue Elements: E U N I C E
    Front -> E
    Back  -> E
    Empty? No
=====
                DEQUEUEING ELEMENTS
=====
Queue Elements: U N I C E
Queue Elements: N I C E
Queue Elements: I C E
Queue Elements: C E
Queue Elements: E
Is Empty? No
```

ANALYSIS:

In the output, it showed that the elements "EUNICE" starts and ends with the same letter, Front and Back both show E right after enqueueing. Each subsequent dequeue removes exactly one letter from the front, confirming the linked-list queue preserves strict FIFO order just like the STL version. Since only 5 of the 6 letters were dequeued in this run, the isEmpty() correctly reports no because one node (E) is still in the list.

OBSERVATION

After building the queue manually with frontPtr and backPtr, it is clear that enqueue and dequeue are just quick pointer updates, not something that requires shifting or rebuilding the whole list. The part that needed the most

attention was removing the very last node, since both frontPtr and backPtr have to be reset together, otherwise one of them would be left pointing to freed memory.

ILO B2

MAIN.CPP

```
#include <iostream>
#include "queueArr.h"

int main(){
    //creating an object
    queueArr<int> q(5);

    //enqueue 1, 2, 3, 4, 5
    q.Enqueue(1);
    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(2);
    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(3);
    q.Enqueue(4);
    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(5);
    std::cout << "Front: " << q.front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(6);

    std::cout << "\nRemoved: " << q.Dequeue() << std::endl;
    std::cout << "Removed: " << q.Dequeue() << std::endl;

    q.Clear();
    std::cout << "Is the Queue Empty? " << (q.Empty() ? "Yes" : "No") << std::endl;

    return 0;
}
```

CODE ANALYSIS:

In this main.cpp source code, we first created an object where queueArr<int> q(5) creates a fixed-capacity circular queue that can hold 5 integers. The program enqueues 1 through 5 one at a time, printing Front(), Back(), and Size() after each insertion to show the queue moving. It then enqueues a 6th element (Enqueue(6)) even though the queue is already full, followed by two Dequeue() calls, a Clear(), and a final Empty() check.

QUEUEARR.H

CODE ANALYSIS:

In this header file, we made the class store its elements in a dynamic array (q_array), along with q_capacity, q_size, q_front, and q_back to track the array's limits and the queue's current state. Before adding an item, Enqueue() checks Full() to make sure there is still room, then moves q_back forward using modulo (% q_capacity) so it would go back to the index 0 once it reaches the end. Dequeue() works the same way in reverse, it moves q_front forward and reduces q_size, and calls Clear() once the queue becomes empty. The copy constructor and assignment operator copy the whole array rather than just the active elements, since the actual data can wrap around from the end of the array back to the beginning.

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

//array Based circularqueue
template <typename T>
class queueArr {
private:
    //pointer to dynamically allocate array
    T* q_array;
    // maximum number of the elements a queue can hold
    size_t q_capacity;
    //current number of the elements in the queue
    size_t q_size;
    //index of the front element
    int q_front;
    //index of the back element
    int q_back;

public:
    //constructor
    queueArr(size_t capacity = 10);
    //copy constructor
    queueArr(const queueArr<T>& other);
    //copy assignment operator
    queueArr& operator=(const queueArr<T>& other);
    //destructor
    ~queueArr();

    //queue operations
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T front();
    T Back();
    void Enqueue(T value);
    T Dequeue();
};

//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    //initialize the capacity, size, front, and back
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    // allocate memory for the array
    q_array = new T[q_capacity];
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr<T>& other){
    //copy the capacity
    q_capacity = other.q_capacity;
    //copy the size
    q_size = other.q_size;
    //copy the front index
    q_front = other.q_front;
    //copy the back index
    q_back = other.q_back;
    //dynamically allocate memory for the array
    q_array = new T[q_capacity];
    //copy the elements from the other queue to this queue
    for (size_t i = 0; i < q_size; i++) {
        q_array[q_front + i] % q_capacity = other.q_array[(other.q_front + i) % other.q_capacity];
    }
}
```

```
//copy assignment operator
template<typename T>
queueArrc(T& queueArrc):operator=(const queueArrc(T& other) {
    if (this != &other) {
        //delete the existing array
        delete[] q_array;
        //copy the capacity
        q_capacity = other.q_capacity;
        //copy the size
        q_size = other.q_size;
        //copy the front index
        q_front = other.q_front;
        //copy the back index
        q_back = other.q_back;
        //dynamically allocate memory for the array
        q_array = new T[q_capacity];
        //copy the elements from the other queue to this queue
        for (size_t i = 0; i < q_size; i++) {
            q_array[(q_front + i) % q_capacity] = other.q_array[(other.q_front + i) % other.q_capacity];
        }
    }
    return *this;
}

//destructor
template<typename T>
queueArrc::~queueArrc() {
    delete[] q_array;
}

//empty()
template<typename T>
bool queueArrc::empty() {
    return q_size == 0;
}

//full()
template<typename T>
bool queueArrc::full() {
    //check if the size reached the maximum capacity
    return q_size == q_capacity;
}

//size()
template<typename T>
size_t queueArrc::size() {
    //find the size of the array
    return q_size;
}

//front()
template<typename T>
T queueArrc::front() {
    //check if the queue is empty
    if (empty()) {
        std::cout << "Queue is empty." << std::endl;
        return T{};
    }
    //return front
    return q_array[q_front];
}

//back()
template<typename T>
T queueArrc::back() {
    //check if the queue is empty
    if (empty()) {
        std::cout << "Queue is empty." << std::endl;
        return T{};
    }
    //return back
    return q_array[q_back];
}

//clear()
template<typename T>
void queueArrc::clear() {
    //reset the q_size, q_front, and q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}
```

```
//Enqueue()
template<typename T>
void queueArr<T>::Enqueue(T value) {
    //check if the queue is full?
    if (Full()) {
        std::cout << "Queue is Full." << std::endl;
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    //store new value to the back
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
}

//T Dequeue()
template<typename T>
T queueArr<T>::Dequeue() {
    //check if empty
    if (Empty()) {
        std::cout << "Queue is Empty." << std::endl;
        return T{};
    }
    //create a temporary variable to store the current front
    T temp = q_array[q_front];
    //move q_front circularly
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    //reset the indexes if the queue becomes empty
    if (Empty()) Clear();
    //return the temporary variable
    return temp;
}

#endif //Queue_h
```

OUTPUT

```
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is Full.

removed: 1
removed: 2
is the queue empty? yes
```

ANALYSIS:

In the output, it showed that the front stays at 1 through every enqueue because nothing has been removed yet, while Back updates to reflect the most recently added value. Once the queue holds 5 elements at capacity 5, the attempt to enqueue a 6th value is rejected by the Full() check, producing "Queue is Full.". Instead of overwriting existing data. After that, two elements (1 and 2) are removed and printed as removed: 1 and removed: 2, which would normally leave 3 elements behind. But the program then calls q.Clear(), which immediately resets q_size to 0 no matter how many elements are actually left. That's why Is the Queue Empty? reports Yes, even though only 2 of the 5 elements were removed through Dequeue(). The key difference: Dequeue() removes items one at a time, while Clear() wipes everything at once.

OBSERVATION

I observed that the circular array version needed the most careful handling out of all three, since q_front and q_back can go back around to index 0 once they pass the end of the array. The Full() check worked correctly

by rejecting the 6th enqueue instead of silently overwriting existing data, showing that the bounds checking was doing its job. This part also made the difference between Dequeue() and Clear() clearer, since Dequeue() removes one item at a time while Clear() resets everything at once regardless of what's still in the queue.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools

Used: Microsoft copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose

of Use: Explanation of concepts, wordings

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Used AI in the google search bar to help me explain my explanation be better like synonyms and how to grammatically say this right

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.